Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Dersin tarihi 16 Mayıs 2023 Salı. Ders Zoom üzerinden yapıldı.
  - Başta birkaç dakika hocanın mikrofonu kapalı kalıyor. Anlatım yaklaşık 00:08'de başlar.
  - Hoca bu dersi "12. haftanın dersi" olarak anıyor. Son teorik ders budur.
- Proje duyuruları
  - Son iki hafta proje sunumları yapılacak. Teslim için bir ödev açılacak.
- Kodlama (gerçekleştirme)
  - Tasarımdan sonra gelir. Kodlama standartları, okunabilirlik, açıklama satırları, anlamlı isimlendirme.
- Programlama dili seçimi ölçütleri.
- Yazılım kalitesi ve kalite faktörleri
  - 11 temel kalite faktörü ve işlevsel olmayan gereksinimlerle ilişkisi.
- Kalite güvencesi
  - Metrikler, ölçüm, kilometre taşlarında gözden geçirme, test planları.
- Test türleri
  - Fonksiyonel, performans, dayanıklılık, yapısal testler.
- Test adımları
  - Birim, bütünleme, onaylama, sistem testi (test spirali).
- Yeni sisteme geçiş
  - Hazırlık: donanım kurulumu, veri aktarımı, eğitim.
  - Beş geçiş yöntemi.
- Bakım
  - İyileştirici, uyarlayıcı, düzeltici bakım. USİS örneği.
- Günümüz yaklaşımları
  - DevOps, CI/CD, mikro servisler, sunucusuz mimari.

Hocanın Özellikle Vurguladığı Kısımlar

- Tasarım olmadan, en azından bir tasarım fikri olmadan koda geçmek çoğu zaman kötü ya da yanlış bir ürüne götürür.
  - Çevik süreçlerde bile küçük bir analiz ve tasarımdan sonra kodlanır.
- Kodlama standartlarına (coding conventions) uyun.
  - Kod kolay okunmalı, kolay düzenlenmeli, kolay değiştirilmeli ve test edilebilir olmalı.
- Her kod için açıklama satırı yazın.
  - Bir hafta sonra kendi kodunuza baktığınızda bile ne yaptığınızı anlamayabilirsiniz.
  - Doğru biçimde yazılan yorumlardan API dokümanı otomatik üretilebilir.
- Değişkenlere anlamlı isim verin.
  - `a`, `b`, `c` gibi isimler kullanmayın. "toplam", "ortalama" gibi çok genel isimler de kullanmayın. Neyin toplamı olduğunu söyleyin.
- Kalite faktörleri ciddi maliyettir ve tasarımı baştan değiştirebilir.
  - "10 000 eş zamanlı kullanıcıda tepki süresi 100 ms'yi geçmesin" gibi bir beklenti bütün mimariyi değiştirir. Dağıtık yapı gerekir.
  - Bu yüzden kalite beklentileri ön incelemede, işlevsel çözümlemeden önce netleştirilmelidir.
- Programlama dili bir araçtır.

  - Amaca uygun aracı seçme sorumluluğu mühendisindir.
- Her geçişte eğitim şarttır.
  - Eğitim üç kesime verilir: sistemle ilgilenecek teknik personel, son kullanıcılar, yönetim kademesi.
- Bir bilgi sisteminin yaşam ömrünün yaklaşık %40–%80'i bakımdır.
- Bu projede stres testi, kalite faktörü testi beklenmiyor.
  - İşlevsel gereksinimleri karşılayan bir sistem geliştirmeniz yeterli.
  - İsterseniz bir modül için birim testi ya da bütünleme/onaylama testi yapıp raporlayabilirsiniz.

Kısa Tekrar Notları

- İyi kodun özellikleri: okunabilir, kolay düzenlenebilir, test edilebilir, kolay değiştirilebilir, açıklamalı, anlamlı isimli.
- Programlama dili seçimi ölçütleri:
  - Dilin uygulama alanı (bilimsel, işlem ağırlıklı, genel amaçlı).
  - Algoritma ve veri yapılarının karmaşıklığı; hazır yapılar.
  - Yazılımın çalışacağı ortam (gömülü platform, derleyici desteği).
  - Uygulamanın çalışma koşulları (dokunmatik, mobil).
  - Ekibin bilgi düzeyi.
  - Lisans ve işletme maliyetleri (uygulama sunucusu lisansı).
  - Müşterinin özel kısıtları ("Bütün sistemlerimiz Java, .NET istemiyoruz").
- Yazılım kalitesi: İşlevsel gereksinimlere, geliştirme standartlarına ve beklenen tüm özelliklere tam uymak.
- 11 kalite faktörü:
  - Doğruluk, güvenilirlik, verimlilik.
  - Güvenlik, kullanılabilirlik, bakım kolaylığı (hata bulma kolaylığı).
  - Esneklik, test edilebilirlik, taşınabilirlik.
  - Yeniden kullanılabilirlik, birlikte çalışabilirlik (bağlanabilirlik).
- Kalite güvencesi planları: yazılım inceleme planı, program sınama (test) planı, kabul muayene planı.
- Doğrulama ve geçerleme:
  - "Ürünü doğru yapıyor muyuz?" (verification).
  - "Doğru ürünü yapıyor muyuz?" (validation).
- Test türleri:
  - Fonksiyonel: girdi–işlem–çıktı, uç değer analizi.
  - Performans: tepki süresi, bellek, iletişim hızı, darboğazlar.
  - Dayanıklılık (stres): aşırı yük, uzun süreli yük, kuyruk ve veri kaybı.
  - Yapısal: kodun iç mantığı, kontrol akışı.
- Test spirali (dıştan içe: sistem mühendisliği → gereksinimler → tasarım → kodlama):

Aşama	Karşılık gelen test
Kodlama	Birim (ünite) testi
Tasarım	Bütünleme (entegrasyon) testi
Gereksinimler	Onaylama (kabul) testi
Sistem geliştirme	Sistem testi (performans, dayanıklılık, güvenlik)

- Geçiş yöntemleri: doğrudan, paralel, dereceli, dağıtılmış, prototip.
- Bakım türleri:
  - İyileştirici: değişen ihtiyaç, yeni özellik.
  - Uyarlayıcı: altyapı değişimi, ör. buluta taşıma.
  - Düzeltici: testte kaçan hatalar.

Detaylı Açıklamalar

Kalite faktörleri ve işlevsel olmayan gereksinimler

Kalite faktörlerinin çoğu işlevsel olmayan gereksinimlerden doğar. “Ders kaydı yapılabilmeli” işlevsel bir gereksinimdir. “10 000 kullanıcı aynı anda bağlandığında tepki süresi 100 ms’yi geçmemeli” işlevsel olmayan bir gereksinimdir. İkincisi, sistemin dağıtık kurulmasını ve servislerin farklı bölünmesini gerektirebilir. Tasarım değişince gerçekleştirme de değişir.

- **Verimlilik:** Kaynakların (disk, bellek, işlemci) etkin kullanımı.
  - Örnek: Aynı kaydı bir sistem 1 MB’ta, başka bir sistem aynı hızla 100 KB’ta tutuyor. İkincisi daha verimlidir.
- Kaliteyi sağlamak için:
  1. Proje planlanırken kalite kontrol yöntemi ve araçları belirlenir.
  2. Metrikler tanımlanır ve süreç boyunca ölçülür.
  3. Kilometre taşlarında (milestone) gözden geçirme yapılır.
  4. Test planı hazırlanır, kod ve süreç test edilir.
- Bunların hepsi zaman ve iş yükü olarak projenin maliyetini artırır.

Test

- Testin amacı hata bulmaktır. Hata bulununca, hatanın yapıldığı aşamaya dönülür:
  - Kod hatası: kod düzeltilir.
  - Tasarım hatası: tasarım değişir, kod güncellenir, tekrar test edilir.
  - Problem yanlış anlaşılmışsa: analize dönülür. Maliyet çok artar.
- Bu yüzden projeyi küçük parçalara bölüp iteratif ilerlemek bugün kabul gören yaklaşımdır.
- Dayanıklılık testi örneği: Sistem 1000 eş zamanlı kullanıcıya göre tasarlandı.
  - Test, bu yükü saatlerce sürdürür ya da 5000–10 000 kullanıcıya çıkarır.
  - Nerede darboğaz oluştuğuna, kuyruk mekanizmasının çalışıp çalışmadığına ve ne kadar veri kaybı olduğuna bakılır.
  - Kampanya dönemlerinde yük patlıyorsa tasarım ölçeklenebilir olmalıdır.
- Test güdümlü geliştirme (test-driven development, TDD): Önce test, sonra kod yazılır. Birçok dilde hazır birim testi kütüphaneleri vardır.

Yeni sisteme geçiş

Hazırlık adımları: 1. Donanımın kurulması ve iletişim altyapısının oluşturulması. 2. Eski kayıtların yeni sisteme aktarılması. - Bu kendi başına küçük bir proje gibi ele alınmalıdır. Örnek: USİS’ten OBS’ye geçişte bazı notlar ve dersler hatalı göründü. 3. Yeni sistemde gereken ek temel verilerin girilmesi. - Örnek: Adreslere coğrafi koordinat eklemek. 4. Eğitim.

Geçiş yöntemleri:

Yöntem	Nasıl?	Artı	Eksi
Doğrudan	Belirli bir anda eski sistem kapatılır, yalnız yeni sistem kullanılır.	Eski sistemle uğraşmak gerekmez; iş bir kez yapılır.	Testte kaçan hata kritik işleri durdurabilir. Risk yüksek.
Paralel	T_0–T_1 arasında iki sistem birlikte tam çalışır.	Yeni sistemde sorun çıkarsa hizmet aksamaz; T_1 ertelenebilir.	Her iş iki kez yapılır. İş yükü iki katı.
Dereceli	Özellik özellik (modül modül) yeni sisteme geçilir. Basitten karmaşığa gidilir.	Bir iş ya eski ya yeni sistemde yapılır, çift iş yok.	İki sistem sürekli entegre olmalı, veri alışverişi yapmalı.
Dağıtılmış	Birim birim (departman departman) yeni sisteme geçilir.	Kademeli; risk bölünür.	Bir süre iki sistem birlikte yaşar.

Yöntem	Nasıl?	Artı	Eksi
Prototip	Eski sistem tam kullanılırken prototip geliştirilir. Olgunlaşınca eski sistem kapatılır.	Prototip başarısız olursa eski sistemle devam edilir.	Prototip geliştirme süresince ek iş yükü.

- Dereceli ile dağıtılmış farkı: Dereceli geçişte özellikler, dağıtılmış geçişte departmanlar yavaş yavaş geçer.
- Geçiş yöntemi fizibilitede, hatta project charter aşamasında planlanmalıdır.
- **USİS → OBS örneği:**
  - Resmî olarak doğrudan geçiş denildi.
  - Aslında USİS bir süre çalışmaya devam etti (paralel).
  - Önce lisans, sonra yüksek lisans, sonra diğer birimler geçti (dağıtılmış).
  - Hoca, "doğrudan geçtik" demenin reklam açısından daha çekici olduğunu söyledi.

Bakım

- Yazılım yaşayan bir organizma gibidir; sürekli değişir.
- **İyileştirici bakım:** İhtiyaç değişir ya da yeni özellik eklenir.
  - Örnek: Not sisteminin daha çok aralıklı bir harf skalasına geçmesi, ağırlıklı ortalama hesabının değişmesi.
- **Uyarlayıcı bakım:** Altyapı değişir.
  - Örnek: Sunucudan küme (cluster) yapısına, oradan buluta geçiş.
- **Düzeltici bakım:** Testte fark edilmeyen hatalar sonradan düzeltilir.
- **USİS örneği:** 2003'te kullanılmaya başlandı, 2022'de OBS'ye geçildi.
  - Yaklaşık 1 yıl geliştirme ve 2 yıl prototip dönemi var. Kalan yaklaşık 16 yıl bakımla geçti.
- Sistem ihtiyacı karşılayamaz hâle gelince döngü yeniden başlar: ön inceleme, fizibilite, analiz, tasarım…
- Günümüzde bu döngü DevOps ve CI/CD ile sürekli hâle getiriliyor. Mikro servisler ve sunucusuz mimariler kullanılıyor.
  - Parçalar küçüldükçe birleştirmek zorlaşır. Parçalar büyüdükçe yapı hantallaşır. Bu bir ödünleşimdir (trade-off).

Proje ve sınav duyuruları

- Son iki hafta proje sunumları yapılacak. Program ders duvarında duyurulacak, bilerek geç açıklanacak.
- Rapor, sunum, kodlar, veritabanı ve bütün belgeler açılacak ödeve yüklenecek. Hoca bunları kurup inceleyebilmeli.
  - Grupların hepsi yüklemesin; grup başına bir temsilci yüklesin.
  - Ders bitene kadar kodları public GitHub reposunda paylaşmayın. Çalışan hâlin bir kopyasını yükleyin.
- Müşteri görüşmesi için Zoom kaydı gerekmiyor. Toplantı tutanağı yazılmalı: ne konuşuldu, ne planlandı, kime hangi sorumluluk verildi.
- Ders sonundaki sohbette hoca, finallerin çevrim içi olacağı yönündeki sözün şaka olduğunu söyledi. Sohbetin hangi dersin finaliyle ilgili olduğu kayıtta net değil; hoca aynı öğrencilere Veri Yapıları dersi de veriyor.

Ders sonrası notlar

- **Güvenilirlik (reliability) tanımı:** Hoca güvenilirliği "belirli yük altında tepki süresinin makul sınırda kalması" diye örnekledi. Bu daha çok verimlilik (performans) faktörüne girer. Güvenilirlik, sistemin belirli koşullarda, belirli bir süre boyunca hatasız çalışmasıdır. Örnek ölçüt: arızalar arası ortalama süre (MTBF).
- **Java birim testi kütüphanesi:** Altyazıda "C-Unit" geçiyor. Java'nın standart birim testi kütüphanesi JUnit'tir.

- **“Bir satır koda iki satır açıklama”:** Bu sabit bir kural değildir. Güncel uygulamada kendini açıklayan kod (anlamlı isimler, kısa fonksiyonlar) ile “neden”i anlatan yorumlar birlikte önerilir. Hocanın asıl vurgusu, açıklamasız kodun okunamaz hâle gelmesidir.
- **Geçiş yöntemi sayısı:** Hoca prototip geçişi “dördüncü ve sonuncu” diye andı. Slaytta beş yöntem var: doğrudan, paralel, dereceli, dağıtılmış ve prototip.
- **Bakım türleri:** Ders üç türü anlattı. Literatürde dördüncü bir tür olarak önleyici (preventive) bakım da sayılır. Önleyici bakım, gelecekteki sorunları önlemek için kodun yeniden düzenlenmesidir.
- **USİS süresi:** Derste “18 sene” dendi. 2003–2021 arası 18, 2003–2022 arası 19 yıldır. Hocanın “19 senenin 16’sı bakım” hesabı ikinci aralığa uyar.
- **Kalite faktörü listesi:** Sayılan 11 faktör McCall’un kalite modelindeki faktörlerle örtüşür. “Güvenlik” faktörü McCall modelinde “bütünlük” (integrity) olarak geçer.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.